Airport TimeTwin — คู่มือส่งต่อโปรเจคฉบับย่อ

เอกสารนี้ใช้คู่กับ data-provenance-th.pdf เพื่อให้คนที่ไม่เคยเห็นโค้ดเข้าใจโปรเจคและสาธิตระบบได้อย่างถูกต้อง

1. โปรเจคนี้คืออะไร

Airport TimeTwin เป็น decision-support prototype สำหรับจำลองสถานการณ์ผู้โดยสารในสนามบิน แล้วเปรียบเทียบมาตรการ เช่น เปิดเลนสำรอง, redirect ผู้โดยสาร หรือใช้มาตรการผสม ก่อนให้ผู้ปฏิบัติงานตัดสินใจ

มันเป็น “ห้องทดลองตัดสินใจ” ไม่ใช่ระบบควบคุมสนามบิน และไม่ควรอ้างว่าตัวเลขคิวเป็นข้อมูลจริงของสุวรรณภูมิ

ลำดับหลักคือ:

> Observe → Predict → Simulate → Compare → Human decision → Measure

2. สิ่งที่ระบบทำในเดโม

1. แสดงสถานการณ์ปัจจุบันและ forecast ที่ +15, +30, +60 นาที
2. จำลองคิวของ Security A/B ด้วยข้อมูลชุดเดียวกัน
3. เปรียบเทียบ No Action, เปิดเลน, redirect และมาตรการผสม
4. ตรวจข้อจำกัด เช่น จำนวนเลน, staff และ redirect cap
5. ให้ operator Accept, Reject หรือ Override พร้อมเหตุผล
6. บันทึก decision และผลลัพธ์จำลองลง audit trail
7. จำลอง gate change และคำนวณ accessible route กับ Safe Time ใหม่

3. ขอบเขตข้อมูล

ป้าย	ความหมาย
REAL_PUBLIC	ข้อมูลสาธารณะจริง ใช้เป็น context/benchmark
SYNTHETIC	ข้อมูล fixture ที่สร้างขึ้นสำหรับเดโม
ASSUMPTION	ค่าที่สมมติ เช่น service rate และ topology
DERIVED	ค่าที่คำนวณจากข้อมูลต้นทาง
SIMULATED	ผลจาก engine เช่น queue, forecast และ recommendation

ข้อมูล operational ในเดโมเป็น SYNTHETIC/SIMULATED เป็นหลัก ไม่มี passenger telemetry จริงของ BKK และไม่มีการสั่งงานอุปกรณ์สนามบิน

4. วิธีรัน

วิธีที่แนะนำ: Docker Compose

ต้องเปิด Docker Desktop ก่อน จากโฟลเดอร์โปรเจคให้รัน:

```
cd '/Users/jswvn/Desktop/missj/my-exp/HACKTHON/sky ict/airport-timetwin'
docker compose --env-file .env -f infra/docker-compose.yml up -d --build
```

จากนั้นเปิด:

- เว็บ: http://localhost:8080
- API docs: http://localhost:8000/docs

หยุดระบบด้วย docker compose --env-file .env -f infra/docker-compose.yml down โดย volume ของฐานข้อมูลจะยังอยู่

วิธีรันแบบไม่ใช้ Docker

```
cd '/Users/jswvn/Desktop/missj/my-exp/HACKTHON/sky ict/airport-timetwin'
./run-local.sh
```

แล้วเปิด http://localhost:5173

5. ลำดับเดโมสำหรับการนำเสนอ

1. เปิด /overview เพื่อดู queue, forecast, risk และเที่ยวบินจำลอง
2. เปิด /operations เพื่อดูสถานการณ์ที่ถูกจัดลำดับความสำคัญ
3. ไป /simulator แล้วเปรียบเทียบ intervention หลายแบบ
4. อธิบายว่า demand และ initial queue ของทุกทางเลือกเหมือนกัน
5. เลือก recommendation หรือปรับ what-if แล้วตรวจ feasibility
6. กด Accept, Reject หรือ Override พร้อมใส่เหตุผล
7. เปิด /app/audit หรือ /performance เพื่อดู decision และผลจำลอง
8. เปิด /journey แล้วกดจำลอง gate change เพื่อดู route และ Safe Time ที่เปลี่ยน
9. เปิด /sources เพื่ออธิบาย provenance และขอบเขตข้อมูล

สำหรับ guided demo ที่ควบคุม step ได้ ให้เปิด /demo

6. โครงสร้างระบบแบบสั้น

```
React/TypeScript frontend
      ↓ JSON + demo session
FastAPI API layer
      ↓
Deterministic simulation engine
      ↓
SQLAlchemy → SQLite (local) / PostgreSQL (Compose)
```

Frontend แสดงผลและรับ interaction ส่วน business rules อยู่ใน API/engine การคำนวณคิวใช้หลัก:

```
queue_next = max(0, queue_current + arrivals - lanes × service_rate)
```

Recommendation ใช้ weighted score ที่เปิดเผยและตรวจ resource constraints ไม่ได้สร้างตัวเลขด้วย LLM

7. Endpoint ที่ควรรู้

Endpoint	หน้าที่
GET /api/overview	สถานการณ์, forecast และ KPI
GET /api/recommendations	ทางเลือกและคะแนน
POST /api/simulation	รัน what-if simulation
POST /api/recommendations/{id}/decision	รับ human decision
GET /api/audit	ประวัติการตัดสินใจ
GET /api/results	ผลก่อน/หลัง intervention
GET /api/passenger/{id}/journey	route และ Safe Time
GET /api/data/catalog	lineage และ provenance

8. สิ่งที่ควรพูดและไม่ควรพูด

ควรพูดว่า:

ระบบใช้ public aviation data เป็น context และใช้ deterministic synthetic scenario
เพื่อสาธิตการเปรียบเทียบมาตรการจัดการคิวอย่างโปร่งใส

ไม่ควรพูดว่า:

- โมเดลทำนายคิวสุวรรณภูมิจากข้อมูลจริงได้แล้ว
- ลดเวลารอได้ตามตัวเลขในเดโมจริง
- ผ่านการ validate กับสนามบินแล้ว
- ใช้ข้อมูล passenger จริงของ BKK

9. เอกสารอ่านต่อ

อ่านตามลำดับนี้:

1. README.md — วิธีรันและภาพรวม
2. data-provenance-th.pdf — ที่มาข้อมูลและ flow แบบละเอียด
3. architecture.md — component และ boundary
4. api.md — request/response ของ API
5. demo-script.md — บทพูดและลำดับนำเสนอ
6. assumptions.md — ค่าที่สมมติและข้อจำกัด
7. deployment.md — การ deploy และ production roadmap

10. คำตอบสั้นสำหรับคนถามว่า “ระบบนี้เชื่อถือได้แค่ไหน”

เชื่อถือได้สูงในฐานะ prototype ที่แสดงกลไกการจำลอง, การเปรียบเทียบทางเลือก, ข้อจำกัดทรัพยากร และ auditability อย่าง reproducible แต่ยังไม่ควรใช้ตัวเลขเป็นผลปฏิบัติการจริงจนกว่าจะมี aggregate checkpoint telemetry จากสนามบิน, calibration, temporal validation, security review และ human-approved pilot

11. ผล validation หลายสถานการณ์

ชุด validation ใหม่มี 8 สถานการณ์, 100 seeds ต่อสถานการณ์ และ 4 เงื่อนไข รวม 3,200 paired episodes ใช้ median เป็นค่าหลักและ P25/P75 แสดงความแปรผัน ทุกเงื่อนไขใช้ scenario และ seed เดียวกัน

ผลรวม BASELINE เทียบกับ ONE_TRUSTX_AI:

- Coordination time: 19.66 → 7.67 วินาที (-61%)
- Reconciliation time: 15.29 → 3.36 วินาที (-78%)
- Risk visibility: 45.88 → 11.93 วินาที (-74% หรือเร็วขึ้นประมาณ 34 วินาที)
- Event traceability: 100%
- Event verification rate: 85.7%
- Delay risk proxy: 1.00 → 0.50 (-50%)

ตัวเลขทั้งหมดเป็น SYNTHETIC สำหรับตรวจความสม่ำเสมอและกลไกการเปรียบเทียบของ prototype
ไม่ใช่ผลการทำงานจริงของสนามบิน ไม่ควรเรียก delay risk ว่าโอกาสล่าช้าจริง หรือเรียก recommendation success ว่า AI accuracy